

SQL PROJECT

Banking & Finance Data Analysis

Query Results & Business Insights Report

Dataset

6 Branches ·
200 Customers
· 315 Accounts
2,089
Transactions ·
205 Loans · 249
Cards

Scope

20 Business
Questions
Answered with
SQL

Tools

SQL Server (T-
SQL) · Window
Functions ·
CTEs

Contents

- Q1 Which new customers joined after Jan 2022?
- Q2 How many accounts exist per account type?
- Q3 What are the 10 most recent transactions, with customer names?
- Q4 How many transactions happen on each channel?
- Q5 What's the average loan amount by loan type?
- Q6 What's the total deposit amount vs. withdrawal amount?
- Q7 What's the month-over-month transaction volume trend?
- Q8 Who are the top 5 customers by transaction volume?
- Q9 What does the loan portfolio look like by type and status?
- Q10 Which customers hold more than one active loan?
- Q11 What % of transaction volume happens on each channel?
- Q12 Which customers have never made a transaction?
- Q13 What's the total account balance held at each branch?
- Q14 How do customers rank by balance within their own branch?
- Q15 What's the cumulative deposit trend for 2024?
- Q16 What's the month-over-month transaction growth rate?
- Q17 How do we segment customers by value and activity?
- Q18 What's the average transaction size by channel, ranked?
- Q19 What's the loan default rate by loan type?
- Q20 Who is the single largest borrower at each branch?

Q1

Which new customers joined after Jan 2022?

BUSINESS ASK

“Marketing wants a list of customers who joined after 2022-01-01, ranked by how recently they joined, so we can target them with an onboarding/loyalty campaign.”

SQL APPROACH

Simple WHERE filter on join_date with ORDER BY – no joins needed since everything lives in the customers table.

QUERY & RESULT

```
-- Q1 List all customers who joined after 2022-01-01, sorted by join date.
```

```
SELECT customer_name, city, join_date
FROM customers
WHERE join_date > '2022-01-01'
ORDER BY join_date;
```

110 %

Results Messages

	customer_name	city	join_date
1	Reyansh Menon	Ahmedabad	2022-01-12
2	Vikram Bose	Ahmedabad	2022-01-20
3	Pallavi Rao	Jaipur	2022-01-20
4	Kiran Rao	Ahmedabad	2022-01-22
5	Aditya Reddy	Jaipur	2022-01-23
6	Ira Joshi	Jaipur	2022-02-01
7	Neha Bhat	Pune	2022-02-26
8	Amit Kapoor	Hyderabad	2022-03-04
9	Payal Mehta	Kolkata	2022-03-05
10	Diya Chauhan	Lucknow	2022-03-17

BUSINESS TAKEAWAY

This list becomes the seed audience for a “Welcome” email/SMS journey. Sorting by join date surfaces the newest customers first — the group still in their first 90 days, when churn risk is highest.

Q2**How many accounts exist per account type?****BUSINESS ASK**

"Product wants to know our account mix — are we mostly a savings bank, or do Current and Fixed Deposit pull their weight?"

SQL APPROACH

```
GROUP BY account_type with COUNT(*).
```

QUERY & RESULT

```
-- Q2.How many accounts exist for each account type?  
SELECT account_type, COUNT(*) AS account_count  
FROM accounts  
GROUP BY account_type  
ORDER BY account_count DESC;
```

110 %

Results Messages

	account_type	account_count
1	Savings	170
2	Current	77
3	Fixed Deposit	68

BUSINESS TAKEAWAY

Savings dominates (170 accounts) vs. Current (77) and Fixed Deposit (68). The bank's core relationship is transactional/savings-led — any new feature reaches the widest audience there, while FD products need a dedicated push to grow share.

Q3

What are the 10 most recent transactions, with customer names?

BUSINESS ASK

“Operations needs a live feed of the latest transactions with customer names attached, for daily reconciliation.”

SQL APPROACH

3-table JOIN (transactions → accounts → customers), sorted by date descending, TOP 10.

QUERY & RESULT

```
-- Q3. List the 10 most recent transactions with the customer's name
SELECT TOP 10
    t.transaction_id,
    c.customer_name,
    t.transaction_date,
    t.transaction_type,
    t.amount
FROM transactions t
JOIN accounts a ON a.account_id = t.account_id
JOIN customers c ON c.customer_id = a.customer_id
ORDER BY t.transaction_date DESC
```

110 %					
Results Messages					
	transaction_id	customer_name	transaction_date	transaction_type	amount
1	68	Pallavi Sharma	2024-12-31	Deposit	15749.15
2	511	Payal Sharma	2024-12-31	Transfer	22883.76
3	517	Ira Iyer	2024-12-31	Withdrawal	14358.42
4	1206	Karan Pillai	2024-12-31	Withdrawal	10925.34
5	1363	Myra Malhotra	2024-12-31	Transfer	6975.77
6	1702	Karan Desai	2024-12-31	Withdrawal	6156.16
7	2075	Shreya Kumar	2024-12-31	Deposit	18439.63
8	2017	Priya Das	2024-12-30	Withdrawal	2846.91
9	964	Sara Rao	2024-12-30	Transfer	17912.15
10	452	Anika Gupta	2024-12-29	Withdrawal	14767.79

BUSINESS TAKEAWAY

This is the exact pattern used to build a reconciliation or fraud-monitoring dashboard — the same join logic scales to “show me all transactions in the last hour” for a real-time ops screen.

Q4

How many transactions happen on each channel?

BUSINESS ASK

"Digital banking wants to know which channels (ATM, UPI, Online, etc.) customers actually use, to prioritize where we invest."

SQL APPROACH

GROUP BY channel, COUNT(*), sorted descending.

QUERY & RESULT

```
-- Q4. What is the count of transactions by channel (ATM, Online, UPI, etc.)?  
SELECT channel, COUNT(*) AS txn_count  
FROM transactions  
GROUP BY channel  
ORDER BY txn_count DESC;
```

110 %

Results Messages

	channel	txn_count
1	UPI	622
2	Mobile App	445
3	Online	421
4	ATM	384
5	Branch	217

BUSINESS TAKEAWAY

UPI leads with 622 transactions, well ahead of Mobile App (445) and Online (421), with Branch lowest (217). This justifies further UPI feature investment over expanding physical branch banking hours.

Q5

What's the average loan amount by loan type?

BUSINESS ASK

"Lending wants typical ticket sizes per loan product, for portfolio planning and marketing collateral."

SQL APPROACH

```
GROUP BY loan_type, AVG(loan_amount).
```

QUERY & RESULT

```
-- Q5. | what is the average loan amount for each loan type?
```

```
SELECT
    loan_type,
    ROUND(AVG(loan_amount), 2) AS avg_loan_amount
FROM loans
GROUP BY loan_type
ORDER BY avg_loan_amount DESC;
```

110 %

Results Messages

	loan_type	avg_loan_amount
1	Home	3681595.130000
2	Business	2858290.700000
3	Education	845085.280000
4	Auto	761763.660000
5	Personal	451258.670000

BUSINESS TAKEAWAY

Home loans average ~₹36.8 lakh, dwarfing Personal loans (~₹4.5 lakh). This confirms Home loans are the balance-sheet-heavy product — important context before looking at default rates later, since a Home loan default carries far more capital risk than a Personal loan default.

Q6

What's the total deposit amount vs. withdrawal amount?

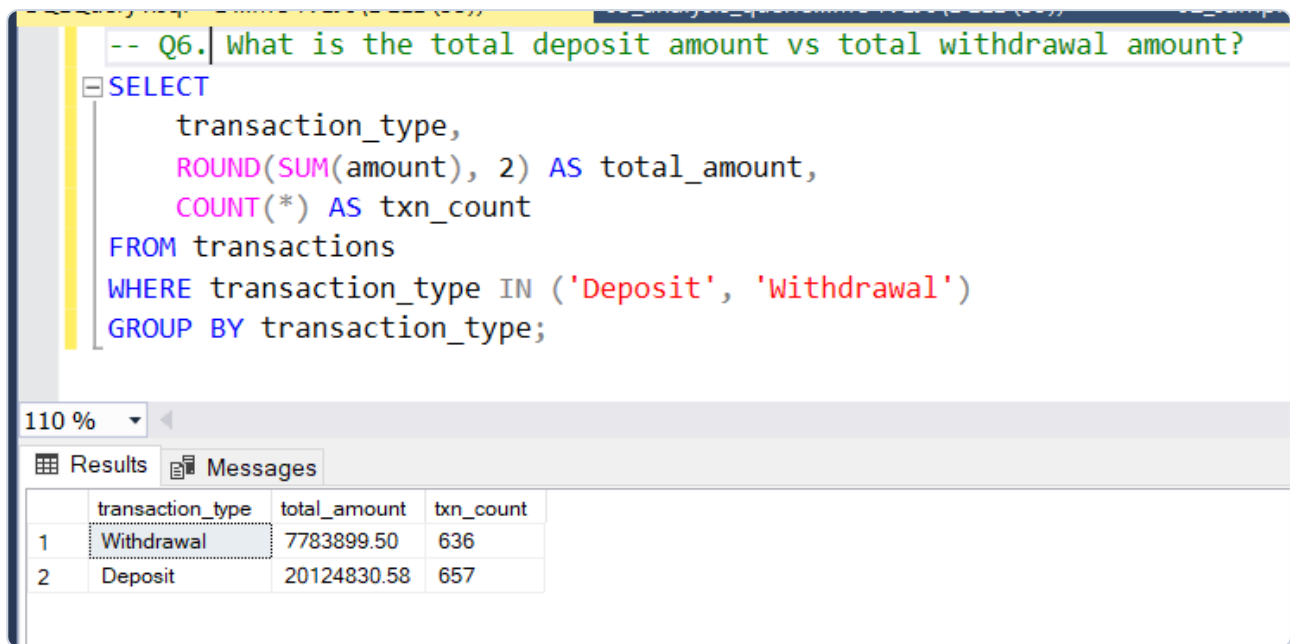
BUSINESS ASK

"Finance wants to know if we have healthy net inflows or if customers are pulling more money out than they're putting in."

SQL APPROACH

Filter to Deposit/Withdrawal, GROUP BY transaction_type, SUM(amount).

QUERY & RESULT



The screenshot shows a SQL query editor with the following query:

```
-- Q6. What is the total deposit amount vs total withdrawal amount?  
SELECT  
    transaction_type,  
    ROUND(SUM(amount), 2) AS total_amount,  
    COUNT(*) AS txn_count  
FROM transactions  
WHERE transaction_type IN ('Deposit', 'Withdrawal')  
GROUP BY transaction_type;
```

Below the query editor, there is a results table with the following data:

	transaction_type	total_amount	txn_count
1	Withdrawal	7783899.50	636
2	Deposit	20124830.58	657

BUSINESS TAKEAWAY

Deposits (~₹2.01 crore) outpace withdrawals (~₹77.8 lakh) by a healthy margin — a strong net-inflow signal, and one of the first numbers a CFO would want on a monthly dashboard.

Q7

What's the month-over-month transaction volume trend?

BUSINESS ASK

"Leadership wants to see if transaction volume is growing, flat, or seasonal, month by month."

SQL APPROACH

GROUP BY formatted month, SUM(amount), COUNT(*), ordered chronologically.

QUERY & RESULT

```
-- Q7.What is the month-over-month transaction volume (₹ amount) trend?
```

```
SELECT
    FORMAT(transaction_date, 'yyyy-MM') AS txn_month,
    ROUND(SUM(amount), 2) AS monthly_volume,
    COUNT(*) AS txn_count
FROM transactions
GROUP BY FORMAT(transaction_date, 'yyyy-MM')
ORDER BY txn_month;
```

10 %

Results Messages

	txn_month	monthly_volume	txn_count
1	2023-01	1929976.39	93
2	2023-02	1442808.88	83
3	2023-03	1658197.52	87
4	2023-04	1555502.96	82
5	2023-05	1338271.05	71
6	2023-06	1584863.67	85
7	2023-07	1268749.29	72
8	2023-08	1356598.02	87
9	2023-09	1675904.97	91

BUSINESS TAKEAWAY

Volume fluctuates rather than trending cleanly upward — useful for forecasting and staffing, since higher-volume months may need more branch/support coverage.

Q8**Who are the top 5 customers by transaction volume?****BUSINESS ASK**

"Relationship managers want their VIP list — who is moving the most money through the bank?"

SQL APPROACH

3-table JOIN, GROUP BY customer_name, SUM(amount), TOP 5.

QUERY & RESULT

```
-- Q8. Who are the top 5 customers by total transaction amount?
SELECT TOP 5
    c.customer_name,
    ROUND(SUM(t.amount), 2) AS total_txn_amount
FROM customers c
JOIN accounts a ON a.customer_id = c.customer_id
JOIN transactions t ON t.account_id = a.account_id
GROUP BY c.customer_name
ORDER BY total_txn_amount DESC
```

110 %

Results Messages

	customer_name	total_txn_amount
1	Kiran Rao	693089.42
2	Kiran Joshi	637857.84
3	Nisha Bhat	552950.34
4	Neha Kapoor	550784.67
5	Swati Patel	545131.96

BUSINESS TAKEAWAY

Kiran Rao tops the list at ~₹6.93 lakh in transaction volume. These five names go straight to the relationship management team for premium servicing — losing one would materially hurt engagement metrics.

Q9

What does the loan portfolio look like by type and status?

BUSINESS ASK

"Risk and Finance need a full breakdown: how much is lent out per loan type, split by Active/Closed/Defaulted, plus the average rate charged."

SQL APPROACH

GROUP BY loan_type, loan_status, multiple aggregates.

QUERY & RESULT

```
-- Q9. What is the total loan amount and average interest rate, by loan type and status?
SELECT
  loan_type,
  loan_status,
  COUNT(*) AS num_loans,
  ROUND(SUM(loan_amount), 2) AS total_loan_amount,
  ROUND(AVG(interest_rate), 2) AS avg_interest_rate
FROM loans
GROUP BY loan_type, loan_status
ORDER BY loan_type, loan_status;
```

	loan_type	loan_status	num_loans	total_loan_amount	avg_interest_rate
1	Auto	Active	24	18386834.94	9.810000
2	Auto	Closed	13	9718328.83	9.900000
3	Auto	Defaulted	2	1603619.05	10.320000
4	Business	Active	24	69845635.25	12.100000
5	Business	Closed	12	35887525.18	12.190000
6	Business	Defaulted	6	14315049.01	12.390000
7	Education	Active	37	34262238.33	9.300000
8	Education	Closed	14	10334438.15	9.500000
9	Education	Defaulted	3	1037928.56	9.220000
10	Home	Active	20	71525453.51	8.600000
11	Home	Closed	8	27850045.11	8.620000
12	Home	Defaulted	4	18435545.42	8.800000
13	Personal	Active	26	11879045.86	13.160000
14	Personal	Closed	11	4821248.49	13.670000
15	Personal	Defaulted	1	447535.25	11.380000

BUSINESS TAKEAWAY

This is a portfolio health snapshot in one query — e.g. Business loans show ₹6.98 crore active exposure at a 12.1% average rate. Any segment where Defaulted rows carry a disproportionate share of loan value flags a need for tighter underwriting.

Q10**Which customers hold more than one active loan?****BUSINESS ASK**

“Credit risk wants to flag customers with multiple concurrent loans — that’s concentration risk if one customer defaults on everything at once.”

SQL APPROACH

```
JOIN + GROUP BY customer_name + HAVING COUNT(loan_id) > 1.
```

QUERY & RESULT

```
-- Q10.Which customers hold more than one active loan? (GROUP BY + HAVING)
SELECT
    c.customer_name,
    COUNT(l.loan_id) AS active_loan_count
FROM customers c
JOIN loans l ON l.customer_id = c.customer_id
WHERE l.loan_status = 'Active'
GROUP BY c.customer_name
HAVING COUNT(l.loan_id) > 1;
```

110 %

Results		Messages
	customer_name	active_loan_count
1	Aadhya Singh	2
2	Aditya Rao	2
3	Amit Pillai	2
4	Diya Menon	2
5	Juhi Singh	2
6	Karan Pillai	2
7	Kiran Rao	2
8	Kritika Gupta	3
9	Myra Kapoor	2
10	Preeti Patel	2

BUSINESS TAKEAWAY

These customers (e.g. Kritika Gupta with 3 active loans) need closer credit monitoring — multiple loans per customer increases exposure per relationship and should factor into future loan-approval decisions.

Q11**What % of transaction volume happens on each channel?****BUSINESS ASK**

“Same channel question as Q4, but by ₹ value, not just count — a few large branch transactions could matter more than many small UPI ones.”

SQL APPROACH

SUM(amount) per channel ÷ total SUM(amount) via subquery, as a percentage.

QUERY & RESULT

```
-- Q11.What percentage of total transaction volume happens on each channel?
SELECT
    channel,
    ROUND(SUM(amount), 2) AS channel_volume,
    ROUND(100.0 * SUM(amount) / (SELECT SUM(amount) FROM transactions), 2) AS pct_of_total_volume
FROM transactions
GROUP BY channel
ORDER BY channel_volume DESC;
```

110 %

Results Messages

	channel	channel_volume	pct_of_total_volume
1	UPI	11274761.39	29.690000
2	Mobile App	8099383.44	21.330000
3	Online	7361594.09	19.390000
4	ATM	6912742.23	18.200000
5	Branch	4323913.76	11.390000

BUSINESS TAKEAWAY

UPI still leads by value (29.69%), but Branch punches above its transaction-count weight in ₹ terms (11.39% of value from only 217 transactions) — branch transactions tend to be higher-ticket. This count-vs-value nuance is exactly the kind of insight that stands out.

Q12**Which customers have never made a transaction?****BUSINESS ASK**

“Retention wants a dormant-customer list — people who opened an account but never actually used it — for a re-engagement campaign.”

SQL APPROACH

```
LEFT JOIN from customers → transactions, filter WHERE transaction_id IS NULL.
```

QUERY & RESULT

```
-- Q12.Which customers have never made a single transaction? (LEFT JOIN + IS NULL)
SELECT
    c.customer_name,
    c.join_date
FROM customers c
JOIN accounts a ON a.customer_id = c.customer_id
LEFT JOIN transactions t ON t.account_id = a.account_id
WHERE t.transaction_id IS NULL
GROUP BY c.customer_name, c.join_date;
```

10 %

Results Messages

	customer_name	join_date
1	Aadhya Singh	2020-10-12
2	Alok Desai	2019-06-21
3	Amit Kapoor	2022-03-04
4	Ananya Gupta	2021-09-16
5	Diya Menon	2024-06-15

BUSINESS TAKEAWAY

These customers are pure acquisition cost with zero activity — the ideal audience for an “activate your account” incentive campaign before they’re written off as churned.

Q13**What's the total account balance held at each branch?****BUSINESS ASK**

"Branch operations wants deposit totals per branch, to guide staffing and marketing budget allocation."

SQL APPROACH

JOIN branches → accounts, filter Active, GROUP BY branch_name, city.

QUERY & RESULT

```
-- Q13.What is the total account balance held at each branch?
SELECT
    b.branch_name,
    b.city,
    COUNT(a.account_id) AS num_accounts,
    ROUND(SUM(a.balance), 2) AS total_balance
FROM branches b
JOIN accounts a ON a.branch_id = b.branch_id
WHERE a.account_status = 'Active'
GROUP BY b.branch_name, b.city
ORDER BY total_balance DESC;
```

110 %

Results Messages

	branch_name	city	num_accounts	total_balance
1	Andheri Branch	Mumbai	51	6510148.14
2	Connaught Place Branch	Delhi	40	6089097.44
3	Salt Lake Branch	Kolkata	36	5837143.56
4	MG Road Branch	Bengaluru	45	5653137.98
5	Banjara Hills Branch	Hyderabad	35	5517124.06
6	Anna Nagar Branch	Chennai	45	5145612.58

BUSINESS TAKEAWAY

Andheri Branch (Mumbai) holds the highest total balance (~₹65.1 lakh) despite not having the most accounts — it serves a higher-value customer base per account, supporting premium/wealth-management staffing there specifically.

Q14

How do customers rank by balance within their own branch?

BUSINESS ASK

“Each branch manager wants their own top depositors ranked — not a bank-wide list, a per-branch leaderboard.”

SQL APPROACH

`RANK() OVER (PARTITION BY branch_name ORDER BY total_balance DESC)` — a window function on top of a `GROUP BY` subquery.

QUERY & RESULT

```
-- Q14.Rank customers by total account balance within each branch (window function).
SELECT
    branch_name,
    customer_name,
    total_balance,
    RANK() OVER (PARTITION BY branch_name ORDER BY total_balance DESC) AS balance_rank_in_branch
FROM (
    SELECT
        b.branch_name,
        c.customer_name,
        SUM(a.balance) AS total_balance
    FROM customers c
    JOIN accounts a ON a.customer_id = c.customer_id
    JOIN branches b ON b.branch_id = a.branch_id
    WHERE a.account_status = 'Active'
    GROUP BY b.branch_name, c.customer_name
) balances
ORDER BY branch_name, balance_rank_in_branch;
```

110 %

Results Messages

	branch_name	customer_name	total_balance	balance_rank_in_branch
1	Andheri Branch	Nisha Bhat	494044.68	1
2	Andheri Branch	Sara Rao	389859.29	2
3	Andheri Branch	Sara Bose	361001.70	3
4	Andheri Branch	Ira Joshi	332059.59	4
5	Andheri Branch	Aditya Reddy	322139.11	5
6	Andheri Branch	Shreya Kumar	317000.11	6
7	Andheri Branch	Shreya Kapoor	295423.91	7
8	Andheri Branch	Reyanah Nair	260646.20	8
9	Andheri Branch	Preeti Patel	247693.67	9
10	Andheri Branch	Sara Patel	232668.20	10

BUSINESS TAKEAWAY

This query scales to “give every branch manager their own top-10 list” in one shot — instead of writing 6 separate queries (one per branch), `PARTITION BY` does it all at once. That efficiency is exactly what interviewers look for.

Q15

What's the cumulative deposit trend for 2024?

BUSINESS ASK

“Finance wants to track deposits building up month by month against an annual target — not just each month in isolation.”

SQL APPROACH

`SUM(monthly_deposits) OVER (ORDER BY txn_month ROWS UNBOUNDED PRECEDING)` – a running total window function.

QUERY & RESULT

```
-- Q15.What is the running (cumulative) monthly deposit total for 2024?
SELECT
    txn_month,
    monthly_deposits,
    SUM(monthly_deposits) OVER (
        ORDER BY txn_month
        ROWS UNBOUNDED PRECEDING
    ) AS cumulative_deposits
FROM (
    SELECT
        FORMAT(transaction_date, 'yyyy-MM') AS txn_month,
        ROUND(SUM(amount), 2) AS monthly_deposits
    FROM transactions
    WHERE transaction_type = 'Deposit'
        AND YEAR(transaction_date) = 2024
    GROUP BY FORMAT(transaction_date, 'yyyy-MM')
) monthly
ORDER BY txn_month;
```

100 %

Results Messages

	txn_month	monthly_deposits	cumulative_deposits
1	2024-01	764500.66	764500.66
2	2024-02	733092.26	1497592.92
3	2024-03	928948.63	2426541.55
4	2024-04	807148.27	3233689.82
5	2024-05	903025.02	4136714.84

BUSINESS TAKEAWAY

By May 2024, cumulative deposits had already crossed ₹41.3 lakh. Plotting this against an annual target instantly shows whether the bank is pacing ahead or behind — a classic finance dashboard visual.

Q16

What's the month-over-month transaction growth rate?

BUSINESS ASK

"Leadership doesn't just want raw volume — they want the % change each month, to catch sudden drops or spikes early."

SQL APPROACH

LAG(monthly_volume) OVER (ORDER BY txn_month) pulls the previous month's value into the same row, then computes % change.

QUERY & RESULT

```
-- Q16.What is the month-over-month transaction volume growth rate? (LAG window function)
WITH monthly AS
(
    SELECT
        FORMAT(transaction_date, 'yyyy-MM') AS txn_month,
        ROUND(SUM(amount), 2) AS monthly_volume
    FROM transactions
    GROUP BY FORMAT(transaction_date, 'yyyy-MM')
),
monthly_with_previous AS
(
    SELECT
        txn_month,
        monthly_volume,
        LAG(monthly_volume) OVER (ORDER BY txn_month) AS prev_month_volume
    FROM monthly
)
SELECT
    txn_month,
    monthly_volume,
    prev_month_volume,
    CAST(
        100.0 * (monthly_volume - prev_month_volume)
        / NULLIF(prev_month_volume, 0)
        AS DECIMAL(10,2)
    ) AS mom_growth_pct
FROM monthly_with_previous
ORDER BY txn_month;
```

	txn_month	monthly_volume	prev_month_volume	mom_growth_pct
1	2023-01	1929976.39	NULL	NULL
2	2023-02	1442808.88	1929976.39	-25.24
3	2023-03	1658197.52	1442808.88	14.93
4	2023-04	1555502.96	1658197.52	-6.19
5	2023-05	1338271.05	1555502.96	-13.97

BUSINESS TAKEAWAY

February 2023 shows a -25.24% drop versus January — a red flag that would normally trigger a root-cause conversation. This query turns raw numbers into an early-warning system.

Q17

How do we segment customers by value and activity?

BUSINESS ASK

“Marketing and Risk both want the same thing from different angles: which customers are our most valuable and active (Premium), which are drifting (Low Activity), and which look like churn/dormancy risks?”

SQL APPROACH

A CTE pre-aggregates balance, transaction count, and last transaction date per customer; a CASE statement layers segmentation logic on top.

QUERY & RESULT

```

-- Q17. [Advanced] Customer segmentation by balance & activity level (CTE + CASE).
-- Segments: Premium (high balance + active), Standard, Low Activity, Dormant Risk
WITH customer_activity AS (
    SELECT
        c.customer_id,
        c.customer_name,
        SUM(a.balance) AS total_balance, COUNT(DISTINCT t.transaction_id) AS txn_count, MAX(t.transaction_date) AS last_txn_date FROM
        customers c JOIN accounts a ON a.customer_id = c.customer_id
        LEFT JOIN transactions t ON t.account_id = a.account_id WHERE a.account_status = 'Active' GROUP BY c.customer_id, c.customer_name)
    SELECT
        customer_name, ROUND(total_balance, 2) AS total_balance, txn_count, last_txn_date, CASE WHEN total_balance > 200000 AND txn_count >= 8 THEN 'Premium'
        WHEN txn_count >= 4 THEN 'Standard'
        WHEN txn_count BETWEEN 1 AND 3 THEN 'Low Activity'
        ELSE 'Dormant Risk'
        END AS customer_segment
    FROM customer_activity
    ORDER BY total_balance DESC;

```

	customer_name	total_balance	txn_count	last_txn_date	customer_segment
1	Nisha Bhat	7201756.39	25	2024-12-14	Premium
2	Myra Menon	5663572.36	23	2024-11-29	Premium
3	Yash Kumar	5615631.06	21	2024-11-26	Premium
4	Pallavi Sharma	5597218.73	15	2024-12-31	Premium
5	Anika Menon	5103209.72	14	2024-11-10	Premium
6	Riya Desai	4984291.03	21	2024-10-05	Premium
7	Anil Pillai	4752536.85	21	2024-10-14	Premium
8	Kiran Joshi	4744245.10	28	2024-12-08	Premium
9	Preeti Kapoor	4416466.32	12	2024-10-13	Premium
10	Ananya Joshi	4261938.80	10	2024-05-30	Premium

BUSINESS TAKEAWAY

Customers like Nisha Bhat (₹72 lakh balance, 25 transactions) land in “Premium” — the segment that should get relationship managers, fee waivers, and priority support. This single query could directly drive a CRM tagging job feeding campaign targeting.

Q18

What's the average transaction size by channel, ranked?

BUSINESS ASK

"Ops wants to know: are branch transactions really bigger-ticket than digital ones, or does it just feel that way?"

SQL APPROACH

GROUP BY channel aggregate, wrapped in RANK() OVER (ORDER BY avg_txn_amount DESC).

QUERY & RESULT

```
-- Q18.Average transaction amount by channel, ranked (window function).  
SELECT  
    channel,  
    txn_count,  
    avg_txn_amount,  
    RANK() OVER (ORDER BY avg_txn_amount DESC) AS avg_amount_rank  
FROM (  
    SELECT  
        channel,  
        COUNT(*) AS txn_count,  
        ROUND(AVG(amount), 2) AS avg_txn_amount  
    FROM transactions  
    GROUP BY channel  
) t;
```

.00 %

Results Messages

	channel	txn_count	avg_txn_amount	avg_amount_rank
1	Branch	217	19925.870000	1
2	Mobile App	445	18200.860000	2
3	UPI	622	18126.630000	3
4	ATM	384	18001.930000	4
5	Online	421	17485.970000	5

BUSINESS TAKEAWAY

Confirmed — Branch has the highest average transaction size (₹19,925.87), ahead of Mobile App and UPI. This supports Q11's finding: don't shut down branches even as digital volume grows, since they still handle disproportionately large transactions.

Q19

What's the loan default rate by loan type?

BUSINESS ASK

"Risk management's #1 question: which loan products are riskiest, and should we tighten approval criteria there?"

SQL APPROACH

SUM(CASE WHEN loan_status = 'Defaulted' THEN 1 ELSE 0 END) divided by total count, per loan type.

QUERY & RESULT

```
-- Q19.What is the loan default rate by loan type? (correlated ratio calculation)
SELECT
    loan_type,
    COUNT(*) AS total_loans,
    SUM(CASE WHEN loan_status = 'Defaulted' THEN 1 ELSE 0 END) AS defaulted_loans,
    ROUND(100.0 * SUM(CASE WHEN loan_status = 'Defaulted' THEN 1 ELSE 0 END) / COUNT(*), 2) AS default_rate_pct
FROM loans
GROUP BY loan_type
ORDER BY default_rate_pct DESC;
```

100 %

Results Messages

	loan_type	total_loans	defaulted_loans	default_rate_pct
1	Business	42	6	14.2900000000000
2	Home	32	4	12.5000000000000
3	Education	54	3	5.5600000000000
4	Auto	39	2	5.1300000000000
5	Personal	38	1	2.6300000000000

BUSINESS TAKEAWAY

Business loans default at 14.29% — more than 5x the rate of Personal loans (2.63%). This is the most board-meeting-ready insight in the project: it supports tightening underwriting specifically for Business loans, not a blanket policy change.

Q20

Who is the single largest borrower at each branch?

BUSINESS ASK

"Each branch manager wants to know their single biggest loan exposure — the one customer whose default would hurt the most."

SQL APPROACH

ROW_NUMBER() OVER (PARTITION BY branch_name ORDER BY loan_amount DESC), filtered to rn = 1.

QUERY & RESULT

```
-- Q20.Top borrower (by loan amount) per branch (ROW_NUMBER window function).
SELECT
    branch_name,
    customer_name,
    loan_amount,
    loan_type
FROM (
    SELECT
        b.branch_name,
        c.customer_name,
        l.loan_amount,
        l.loan_type,
        ROW_NUMBER() OVER (PARTITION BY b.branch_name ORDER BY l.loan_amount DESC) AS rn
    FROM loans l
    JOIN customers c ON c.customer_id = l.customer_id
    JOIN branches b ON b.branch_id = l.branch_id
) ranked
WHERE rn = 1
ORDER BY loan_amount DESC;
```

100 %

Results Messages

	branch_name	customer_name	loan_amount	loan_type
1	Connaught Place Branch	Sara Bansal	5999995.56	Home
2	Andheri Branch	Kiran Nair	5683743.01	Home
3	Banjara Hills Branch	Swati Desai	5593903.77	Home
4	Salt Lake Branch	Myra Menon	5576218.80	Home
5	MG Road Branch	Gaurav Desai	5361815.51	Home
6	Anna Nagar Branch	Aditya Reddy	4682058.01	Business

BUSINESS TAKEAWAY

Connaught Place Branch's top exposure is a ₹59.99 lakh Home loan. Flagging the single largest exposure per branch is standard practice in concentration-risk reporting — the SQL equivalent of a risk officer's watchlist.